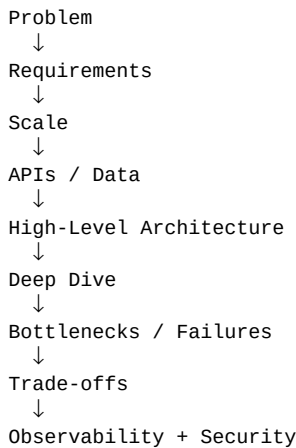


Complete System Design Interview Framework

Hinglish • A repeatable framework for L1/L2/Lead Engineer system design interviews

1. System Design Interview ka Actual Goal

Interviewer usually ye check karta hai ki aap ambiguous problem ko structured engineering problem mein convert kar sakte ho ya nahi. Sirf architecture components ya definitions yaad hona enough nahi.



2. 30-Second Opening

Interview start hote hi directly database ya Redis par jump mat karo. Pehle problem ko frame karo.

"First I'll clarify the functional and non-functional requirements, estimate scale, define the main APIs/data model, then propose a high-level architecture. After that I'll deep-dive into the highest-risk components and discuss scaling, failure handling, observability and trade-offs."

3. Step 1 — Requirements

Requirements ko two buckets mein divide karo: functional and non-functional.

4. Functional Requirements

- User kya action karega?
- System kya output dega?
- Kaunse core workflows hain?
- Read vs write operations kya hain?
- Real-time requirement hai?
- Notifications/analytics required hain?

5. Non-Functional Requirements

- Scale / throughput.
- Latency.
- Availability.
- Consistency.
- Durability.
- Security.
- Cost.

- Data retention.

6. Clarifying Questions

Before designing, ask:

1. How many users?
2. DAU/MAU?
3. Peak RPS?
4. Read/write ratio?
5. Data size?
6. Latency target?
7. Availability target?
8. Strong or eventual consistency?
9. Real-time required?
10. Global or single region?
11. Retention period?
12. Any compliance/security constraints?

7. Don't Ask 20 Questions

Interview ko interrogation mat banao. 3–6 high-value assumptions clarify karo. Missing numbers ko reasonable assumption ke saath state karke proceed karo.

8. Step 2 — Capacity Estimation

Exact numbers se zyada important hai order of magnitude aur bottleneck thinking.

9. Users → RPS

Example:

10 million registered users
DAU = 20%
= 2 million DAU

Each active user:
20 requests/day

Daily requests:
 $2M \times 20$
= 40M requests/day

Average RPS:
 $40M / 86,400$
 ≈ 463 RPS

Peak factor = 5
Peak RPS $\approx 2,315$ RPS

10. Read / Write Split

Peak RPS = 2,500

Read : Write = 90 : 10

Reads $\approx 2,250$ RPS
Writes ≈ 250 RPS

Read-heavy system mein caching, read replicas aur search indexes more relevant ho sakte hain.

11. Storage Estimation

1M new records/day
Average record = 2 KB

Daily:
 $1M \times 2 \text{ KB} = 2 \text{ GB/day}$

1 year:
 $\approx 730 \text{ GB raw}$

Then consider:
indexes + replicas + metadata + backups

12. Bandwidth

RPS $\times$ average response size

$1000 \text{ RPS} \times 50 \text{ KB}$
= 50 MB/s outgoing payload

Then consider:
peak traffic
compression
CDN/cache

13. Connection Estimation

Aapke known production debugging pattern ka use karo: application instances $\times$ pool size can become a DB bottleneck.

10 Node instances
 $\times 20 \text{ DB connections}$
= up to 200 connections

If DB supports 100:
→ connection exhaustion risk

14. Step 3 — Define APIs

```
POST /orders
GET  /orders/:id
POST /orders/:id/cancel
```

Request:

```
{
  productId,
  quantity
}
```

Response:

```
{
  orderId,
  status
}
```

API design mein pagination, idempotency, auth, errors aur versioning ko relevant hone par mention karo.

15. API Design Checklist

- HTTP method.
- Path.
- Request body/query/path params.
- Response.
- Status codes.
- Authentication.
- Idempotency for side-effecting requests.
- Pagination for large lists.

16. Pagination

```
GET /products?limit=20&cursor=abc
```

For deep/high-scale pagination:
cursor/search_after

Avoid:
offset=1000000
when it becomes expensive.

Exact approach DB/search engine par depend karta hai.

17. Step 4 — Data Model

Only relevant entities define karo. Interview mein every column explain karne ki zaroorat nahi.

```
User
-----
id
name
created_at
```

```
Order
-----
id
user_id
status
amount
created_at
```

```
OrderItem
-----
order_id
product_id
quantity
```

18. SQL vs NoSQL

SQL	NoSQL
Strong relational model	Flexible/document/key-value models
Transactions/joins strong	Often horizontal scaling patterns
Structured schema	Schema flexibility depends on DB
Good for transactional systems	Good for suitable high-scale access patterns

Database choice requirement se derive karo; 'NoSQL always scales better' ya 'SQL cannot scale' type blanket statements avoid karo.

19. Indexing

```
Query:
WHERE user_id = ?
ORDER BY created_at DESC
```

Possible composite index:
(user_id, created_at)

Index query pattern ke according design karo. Index read performance improve karta hai but storage/write/maintenance cost add karta hai.

20. Read Replica

```
Primary
  ↓ writes
Replica 1
Replica 2
  ↓ reads
```

Caution:
replication lag
→ stale reads

Read-after-write sensitive requests primary par route karne ka option consider karo.

21. Partitioning

Large logical table ko partitions mein divide kar sakte hain, commonly time/range/hash/list based. Partition key query patterns ke saath align honi chahiye.

```
Transactions
■■■ Jan
```

■■■ Feb
■■■ Mar
■■■ ...

22. Sharding

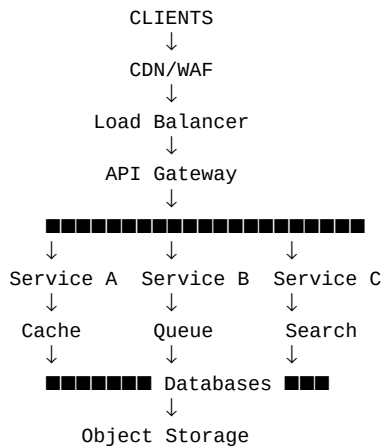
Sharding data ko multiple database nodes/shards mein horizontally distribute karta hai. Shard key critical hai because poor distribution hot partition create kar sakta hai.

userId hash

↓

Shard 1
Shard 2
Shard 3
Shard 4

23. Step 5 — High-Level Architecture



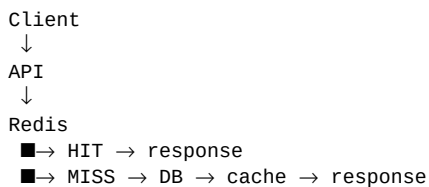
Observability:
Metrics + Logs + Traces

24. Component Selection Cheat Sheet

Requirement	Typical component
Static/media storage	S3/Object Storage
Global static delivery	CDN
Traffic distribution	Load Balancer
API boundary	API Gateway
Fast shared state/cache	Redis
Async processing	Queue/Broker
Full-text search	Elasticsearch/OpenSearch
Transactional source of truth	SQL DB
Large flexible documents	Suitable NoSQL DB
Background processing	Workers
Monitoring	Prometheus/Grafana
Distributed tracing	OpenTelemetry + tracing backend

25. Step 6 — Cache

Cache tab add karo jab access pattern justify kare. Sab kuch Redis mein daalna architecture nahi hai.

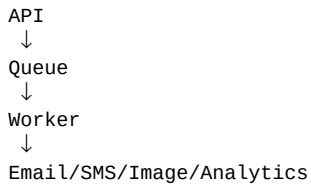


26. Cache Checklist

- Cache key.
- TTL.
- Invalidation.
- Cache-aside vs other strategy.
- Cache stampede.

- Hot keys.
- Redis failure behavior.

27. Step 7 — Async Processing

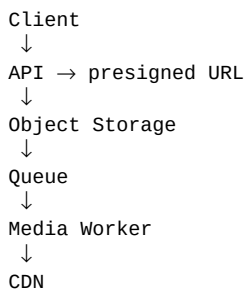


User-facing request ko unnecessarily slow karne wale non-critical work ko async kiya ja sakta hai.

28. Queue Checklist

- Retry.
- Backoff + jitter.
- DLQ.
- Idempotency.
- Ordering requirement.
- Queue depth/lag.
- Worker scaling.

29. Step 8 — File / Media



Large files ko application server ke through proxy karna avoid karna often better hota hai.

30. Step 9 — Rate Limiting

Client
↓
WAF/Gateway
↓
Rate Limiter
↓
API

429 if exceeded

Rate limit identity, endpoint, tenant or API key ke basis par define ho sakti hai. Distributed enforcement ke liye shared state such as Redis use ho sakta hai.

31. Step 10 — Search

Primary DB
↓
CDC / Outbox
↓
Queue
↓
Indexer
↓
Search Index

User
↓
Search API
↓
Search Engine

DB source of truth; search index optimized retrieval representation.

32. Step 11 — Real-Time

Real-time requirement ho to WebSocket/SSE and connection management consider karo.

Client
↕ WebSocket
Gateway / LB
↕
WS Servers × N
↕
Redis / Broker
↕
DB

33. Step 12 — Payment

Payment system mein idempotency, state machine, webhook verification and reconciliation critical concepts hain.

Order
↓
Payment PENDING
↓
Gateway
↓
Webhook/status
↓
SUCCESS / FAILED

Unknown timeout ≠ automatically FAILED

34. Step 13 — Distributed Transactions

Order
↓
Inventory
↓
Payment
↓

Notification

Failure?

- Saga
- compensation
- idempotency
- Outbox

35. Step 14 — Consistency

Business requirement se decide karo:

Need latest value immediately?
→ stronger/read-after-write

Stale data acceptable?
→ eventual consistency

Partition occurs:
→ define CP/AP trade-off

36. Step 15 — Failure Handling

Every dependency:

timeout
retry?
backoff + jitter
circuit breaker
fallback?
idempotency
observability

Important: Retry every failure blindly mat karo. Side effects aur retry storms ko consider karo.

37. Failure Scenario Framework

Interviewer agar bole 'What if X fails?' to ye sequence follow karo.

1. Detect
2. Contain
3. Recover
4. Avoid data corruption
5. Retry/replay if safe
6. Observe
7. Prevent recurrence

38. Dependency Failure

Service A → Service B

B down

A:
timeout
↓
circuit breaker
↓
fallback/pending
↓
metric + alert

39. Database Failure

- Can request be served from cache?
- Read replica available?
- Failover mechanism?
- What writes happen during failover?
- Data consistency/reconciliation?

40. Redis Failure

Decide per use case whether to fail-open, fail-closed, bypass cache and hit DB, or reject. Rate limiting/security and caching have different failure trade-offs.

41. Queue Failure

Producer
↓
Queue unavailable

Options:
retry
local/outbox persistence
degrade
reject

Need:
avoid losing critical event

Outbox is particularly useful when DB state and event publication must remain reliably connected.

42. Region Failure

Region A
↓
Region B

Questions:
active-active?
active-passive?
DNS/global LB?
data replication?

RPO?
RTO?

43. RPO / RTO

RPO	How much recent data loss is acceptable?
RTO	How long can recovery take?

44. Step 16 — Security

- Authentication.
- Authorization.
- TLS.
- Secrets management.
- Rate limiting.
- Input validation.
- Least-privilege IAM/RBAC.
- Tenant isolation.
- Audit logging.
- Encryption at rest where required.

45. Step 17 — Observability

Golden Signals:

1. Latency
2. Traffic
3. Errors
4. Saturation

Also:

logs
metrics
traces
queue lag
DB latency
cache hit ratio

46. p50 / p95 / p99

Average ko alone mat use karo. p95/p99 tail latency ko expose karte hain.

Typical:

p50 = 50 ms

p95 = 180 ms

p99 = 500 ms

Interpretation:

most requests fast,

but tail is hurting some users.

Interview mein 'p99 is bad' mat bolo without context. Target workload, SLO and user experience ke according interpret karo.

47. Bottleneck Identification

High latency



Which endpoint?



RPS changed?



CPU / Memory?



Event Loop?



DB?



Redis?



Queue?



External API?



Trace



Bottleneck

48. Step 18 — Scaling

Vertical:

more CPU/RAM

Horizontal:

more instances

Database:

index

read replicas

partition

shard

Application:

cache

queue

async workers

Edge:

CDN

WAF

rate limit

49. Scaling Decision Tree

Slow?



Optimize query/index?



Cache?



Async?



Scale app?

↓
Scale DB?
↓
Partition/shard?
↓
Architecture change?

50. Step 19 — Trade-offs

Senior/Lead interview mein trade-offs explicitly bolo.

Example:

Redis cache:

- + lower latency
- + lower DB load
- stale data
- invalidation complexity
- memory cost

Read replicas:

- + read scale
- replication lag
- failover complexity

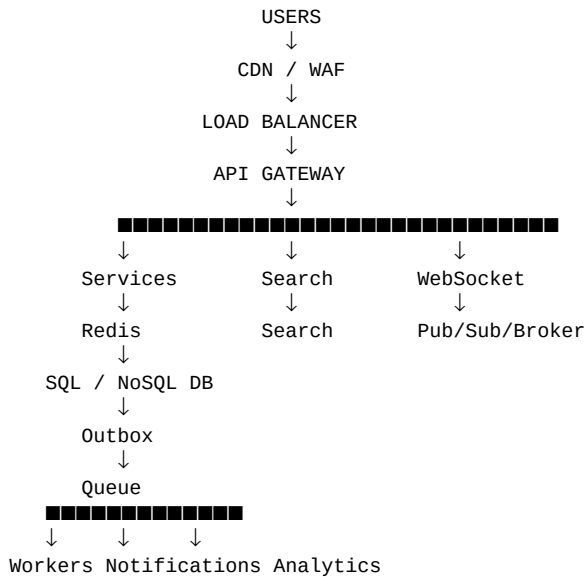
Microservices:

- + independent scaling/deployment
- network + operational complexity

51. Never Say 'Best Architecture'

Instead say: 'Given these requirements, I'd choose X because..., and the trade-off is...'. System design generally context-dependent hota hai.

52. Step 20 — Final Architecture



Object Storage + CDN

Prometheus + Grafana + Traces

53. How to Draw in Interview

Diagram ko left-to-right rakho. Pehle 6–10 boxes enough hain. Deep dive mein boxes add karo.

```
Client
↓
LB/Gateway
↓
API
↓
Cache
↓
DB
↓
Queue
↓
Workers
```

Then:
Search / Object Storage / Observability

54. 45-Minute Interview Time Management

Time	What to do
0–5 min	Requirements + assumptions
5–10 min	Capacity estimation
10–18 min	APIs + data model
18–25 min	High-level architecture
25–35 min	Deep dive bottleneck
35–40 min	Failures + scaling + consistency
40–43 min	Security + observability
43–45 min	Trade-offs + summary

55. If Interviewer Says 'Go Deeper'

Deep dive ko random component par mat le jao. Highest-risk/highest-scale component choose karo.

High traffic?
→ caching/scaling

High writes?
→ DB/queue

Large files?
→ object storage

Search?
→ inverted index/search cluster

Real-time?
→ WebSocket/pub-sub

Money?
→ idempotency/reconciliation

Cross-service workflow?
→ Saga/Outbox

56. Common Mistakes

- Requirements clarify kiye bina architecture banana.
- Numbers estimate na karna.
- Har problem mein Redis/Kafka/Kubernetes add kar dena.

- Single DB ko unlimited capacity assume karna.
- Retries without idempotency.
- Read replicas without discussing lag.
- Cache without invalidation/TTL.
- Microservices without service boundaries.
- Monitoring ko end mein completely ignore karna.
- Trade-offs explain na karna.

57. Golden System Design Vocabulary

RPS
 QPS
 p50 / p95 / p99
 SLA / SLO / SLI
 RPO / RTO
 CAP
 Replication
 Partitioning
 Sharding
 Indexing
 Caching
 CDN
 Load Balancer
 API Gateway
 Queue
 Backpressure
 Idempotency
 Outbox
 Saga
 Circuit Breaker
 Rate Limiter
 WebSocket
 Search Index
 Observability
 Horizontal Scaling

58. Problem → Tool Mapping

Problem	Think about
High read traffic	Cache + read replicas + CDN
High write traffic	DB optimization + partition/shard + queue
Traffic spikes	LB + autoscaling + queue + rate limit
Large files	Object storage + presigned URL + CDN
Full-text search	Search engine + inverted index
Async work	Queue + workers
Real-time	WebSocket + connection layer + broker
Duplicate requests	Idempotency
DB/event inconsistency	Outbox
Multi-service workflow	Saga
Dependency failure	Timeout + circuit breaker
API abuse	Rate limiter + WAF
Production debugging	Logs + metrics + traces

59. Real Interview Practice — URL Shortener

Requirements:
create short URL
redirect
optional custom alias
analytics

Scale:
read-heavy

Architecture:
Client
↓
LB/Gateway
↓
URL Service
↓
Redis
↓
DB

Analytics:
Redirect
↓
Queue
↓
Analytics worker

Deep dive: Base62 ID, collision handling, Redis hot keys, 301/302 choice, rate limiting and analytics async processing.

60. Real Interview Practice — Notification System

Event
↓
Queue
↓
Notification Dispatcher
■→ Email worker
■→ SMS worker
■→ Push worker
↓
Providers

Need:
retry
DLQ
idempotency
provider limits
preferences
status

61. Real Interview Practice — Chat

Client
↑↓ WebSocket
LB/Gateway
↑↓
WS servers × N
↑↓
Redis/Broker
↑↓
Message DB

Need:
presence
heartbeat
ordering
offline sync
read receipts
idempotency

62. Real Interview Practice — Payment

Client
↓
Order
↓
Payment Service
↓
Idempotency
↓
Gateway
↓
Webhook
↓
Payment state
↓
Outbox
↓
Queue

Need:
reconciliation
refunds
duplicate protection

63. Real Interview Practice — Tatkal / Ticket Booking

Users
↓
CDN/WAF
↓
Rate Limit
↓
Gateway
↓
Booking Service
↓
Inventory/Seat Lock
↓
DB/Redis
↓
Payment
↓
Queue

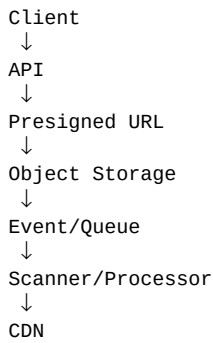
Critical:
concurrency
overselling
idempotency
timeouts
hot keys
load shedding

64. Real Interview Practice — Search

DB
↓
CDC/Outbox
↓
Queue
↓
Indexer
↓
Search Engine

Query:
Client
↓
Search API
↓
Cache
↓
Search Engine
↓
Filters + Ranking

65. Real Interview Practice — File Upload



66. Lead Engineer Answer Structure

1. Clarify
2. Quantify
3. Decompose
4. Design
5. Deep dive
6. Identify bottleneck
7. Handle failure
8. Explain trade-offs
9. Observe
10. Summarize

Lead-level signal ye hai ki aap sirf components nahi, failure modes, operational cost, ownership, rollout, observability aur trade-offs bhi discuss karte ho.

67. 10 Golden Questions to Ask Yourself

1. What is the source of truth?
2. What is the hottest read/write path?
3. What is the peak RPS?
4. What happens when this dependency is slow?
5. What happens when it is down?
6. Can this operation be duplicated?
7. Can this work be asynchronous?
8. Where will data grow fastest?
9. How will I know the system is unhealthy?
10. What trade-off am I making?

68. Final Interview Cheat Sheet

SYSTEM DESIGN

REQUIREMENTS



SCALE ESTIMATION



API + DATA MODEL



HIGH-LEVEL ARCHITECTURE



DB + INDEX



CACHE



QUEUE / ASYNC



CDN / STORAGE / SEARCH / WS



SCALING
↓
CONSISTENCY
↓
FAILURE HANDLING
↓
SECURITY
↓
OBSERVABILITY
↓
TRADE-OFFS
↓
FINAL SUMMARY

69. The Ultimate Golden Answer

"I'll start by clarifying the functional and non-functional requirements and estimate peak traffic, storage and read/write patterns. Then I'll define the APIs and data model and draw a simple high-level architecture. I'll identify the likely bottleneck and deep-dive into the database, cache, queue, search or real-time layer as needed. For scale I'll consider caching, replication, partitioning, sharding and horizontal scaling. For reliability I'll define timeouts, retries, idempotency, circuit breakers and failure recovery. Finally, I'll cover security, observability, deployment strategy and the trade-offs behind the design."

70. Final 80/20

Don't memorize 100 architectures.

Memorize this:

WHAT?

→ requirements

HOW MUCH?

→ scale

WHERE DATA?

→ DB

HOW FAST?

→ cache/index/CDN

HOW ASYNC?

→ queue/workers

HOW SCALE?

→ horizontal/replica/shard

WHAT IF FAILS?

→ timeout/retry/circuit/idempotency

WHAT IF DATA SPLITS?

→ consistency/CAP/Saga

HOW DEBUG?

→ logs/metrics/traces

WHY THIS DESIGN?

→ trade-offs